K. J. Somaiya College of Engineering, Mumbai-77

Batch: A2 Roll No.: 16010324044
Experiment / assignment / tutorial No. 6
Grade: AA / AB / BB / BC / CC / CD /DD

Pr 28/9/26
Signature of the Staff In-charge with date

TITLE: Memory Management
Objective: To allocate virtual memory requirements for processes.
OUTCOME: Student will be able to Understand different approaches to memory management.

Memory Management

- The first-fit, best-fit, or worst-fit strategy is used to select a free hole from the set of available holes.

First fit

- **Allocate the first hole that is big enough.**
- **Searching starts from the beginning of set of holes.**

Algorithm
1. Declare structures *hole* and *process* to hold information about set of holes and processes respectively.
2. Get number of holes, say *nh*.
3. Get the size of each hole
4. Get number of processes, say *np*.
5. Get the memory requirements for each process.
6. Allocate processes to holes, by examining each hole as follows:

a. If hole size > process size then
i. Mark process as allocated to that hole.
ii. Decrement hole size by process size.

b. Otherwise check the next from the set of hole
7. Print the list of process and their allocated holes or unallocated status.
8. Print the list of holes, their actual and current availability.
9. Stop

Department of Electronics and Telecommunication Engineering

OS/Sem V/July.- Nov 26 Page No

TITLE: Memory Management

Objective: To allocate virtual memory requirements for processes.

OUTCOME: Student will be able to Understand different approaches to memory management.

Memory Management

 The first-fit, best-fit, or worst-fit strategy is used to select a free hole from the set of available holes.

First fit

 Allocate the first hole that is big enough.

 Searching starts from the beginning of set of holes.

Algorithm

1. Declare structures *hole* and *process* to hold information about set of holes and processes respectively.
2. Get the number of holes, say *nh*.
3. Get the size of each hole
4. Get the number of processes, say *np*.
5. Get the memory requirements for each process.
6. Allocate processes to holes, by examining each hole as follows:

a. If hole size > process size then
i. Mark process as allocated to that hole.
ii. Decrement hole size by process size.

b. Otherwise check the next from the set of hole
7. Print the list of process and their allocated holes or unallocated status.
8. Print the list of holes, their actual and current availability.
9. Stop

Code:

```
#include <stdio.h>


struct Hole {
```

```
    int id;
    int original_size;
    int remaining_size;
};

struct Process {
    int id;
    int size;
    int allocated;
    int hole_id;
};

int main() {
    int nh, np, i, j;

    struct Hole holes[20];
    struct Process processes[20];

    // Input number of holes
    printf("Enter number of holes: ");
    scanf("%d", &nh);

    // Input hole sizes
    for (i = 0; i < nh; i++) {
        holes[i].id = i + 1;
```

```
    printf("Enter size of Hole %d: ", i + 1);
    scanf("%d", &holes[i].original_size);

    holes[i].remaining_size = holes[i].original_size;
}

// Input number of processes
printf("\nEnter number of processes: ");
scanf("%d", &np);

// Input process sizes
for (i = 0; i < np; i++) {
    processes[i].id = i + 1;

    printf("Enter memory requirement of Process %d: ", i + 1);
    scanf("%d", &processes[i].size);

    processes[i].allocated = 0;
    processes[i].hole_id = -1;
}

// First Fit allocation
for (i = 0; i < np; i++) {

    for (j = 0; j < nh; j++) {
```

```
        // Find the first hole large enough
        if (holes[j].remaining_size >= processes[i].size) {

            processes[i].allocated = 1;
            processes[i].hole_id = holes[j].id;

            // Reduce remaining hole size
            holes[j].remaining_size -= processes[i].size;

            break;
        }
    }
}

// Display allocation
printf("\n----- FIRST FIT ALLOCATION -----\n");

printf("\nProcess\tSize\tHole\tStatus\n");

for (i = 0; i < np; i++) {

    if (processes[i].allocated) {
        printf("P%d\t%d\tH%d\tAllocated\n",
            processes[i].id,
            processes[i].size,
            processes[i].hole_id);
```

```
        }
        else {
            printf("P%d\t%d\t-\tNot Allocated\n",
                processes[i].id,
                processes[i].size);
        }
    }

    // Display hole information
    printf("\n----- HOLE INFORMATION -----\n");

    printf("Hole\tOriginal\tRemaining\n");

    for (i = 0; i < nh; i++) {
        printf("H%d\t%d\t\t%d\n",
            holes[i].id,
            holes[i].original_size,
            holes[i].remaining_size);
    }

    return 0;
}
```

Output:

```
Enter number of holes: 5
Enter size of Hole 1: 100
Enter size of Hole 2: 500
Enter size of Hole 3: 200
Enter size of Hole 4: 300
Enter size of Hole 5: 600
```

```
Enter number of processes: 4
Enter memory requirement of Process 1: 212
Enter memory requirement of Process 2: 417
Enter memory requirement of Process 3: 112
Enter memory requirement of Process 4: 426
```

```
----- FIRST FIT ALLOCATION -----

Process Size    Hole    Status
P1  212 H2  Allocated
P2  417 H5  Allocated
P3  112 H2  Allocated
P4  426 -   Not Allocated
```

```
----- HOLE INFORMATION -----
Hole    Original    Remaining
H1  100     100
H2  500     176
H3  200     200
H4  300     300
H5  600     183
```

Best fit

 Allocate the smallest hole that is big enough.

 The list of free holes is kept sorted according to size in ascending order.

 This strategy produces smallest leftover holes

Algorithm

1. Declare structures *hole* and *process* to hold information about set of holes and processes respectively.
2. Get the number of holes, say *nh*.
3. Get the size of each hole
4. Get the number of processes, say *np*.
5. Get the memory requirements for each process.
6. Allocate processes to holes, by examining each hole as follows:
a. Sort the holes according to their sizes in ascending order
b. If hole size > process size then
i. Mark process as allocated to that hole.
ii. Decrement hole size by process size.
c. Otherwise check the next from the set of sorted hole
7. Print the list of process and their allocated holes or unallocated status.
8. Print the list of holes, their actual and current availability.
9. Stop

Code:

```
#include <stdio.h>


struct Hole {

   int id;

   int original_size;

   int remaining_size;

};


struct Process {

   int id;

   int size;
```

```
    int allocated;
    int hole_id;
};

int main() {
    int nh, np, i, j;

    struct Hole holes[20];
    struct Process processes[20];
    struct Hole temp;

    // Input number of holes
    printf("Enter number of holes: ");
    scanf("%d", &nh);

    // Input hole sizes
    for (i = 0; i < nh; i++) {
        holes[i].id = i + 1;

        printf("Enter size of Hole %d: ", i + 1);
        scanf("%d", &holes[i].original_size);

        holes[i].remaining_size = holes[i].original_size;
    }

    // Input number of processes
```

```
printf("\nEnter number of processes: ");
scanf("%d", &np);

// Input process sizes
for (i = 0; i < np; i++) {
   processes[i].id = i + 1;

   printf("Enter memory requirement of Process %d: ", i + 1);
   scanf("%d", &processes[i].size);

   processes[i].allocated = 0;
   processes[i].hole_id = -1;
}

// Sort holes in ascending order
for (i = 0; i < nh - 1; i++) {
   for (j = i + 1; j < nh; j++) {

      if (holes[i].remaining_size > holes[j].remaining_size) {
         temp = holes[i];
         holes[i] = holes[j];
         holes[j] = temp;
      }
   }
}
```

```
// Best Fit allocation
for (i = 0; i < np; i++) {

   for (j = 0; j < nh; j++) {

      if (holes[j].remaining_size >= processes[i].size) {

         processes[i].allocated = 1;
         processes[i].hole_id = holes[j].id;

         holes[j].remaining_size -= processes[i].size;

         break;
      }
   }
}

// Display allocation
printf("\n----- BEST FIT ALLOCATION -----\n");

printf("\nProcess\tSize\tHole\tStatus\n");

for (i = 0; i < np; i++) {

   if (processes[i].allocated) {
      printf("P%d\t%d\tH%d\tAllocated\n",
```

```
                processes[i].id,
                processes[i].size,
                processes[i].hole_id);
        }
        else {
            printf("P%d\t%d\t-\tNot Allocated\n",
                processes[i].id,
                processes[i].size);
        }
    }

    // Display hole information
    printf("\n----- HOLE INFORMATION -----\n");

    printf("Hole\tOriginal\tRemaining\n");

    for (i = 0; i < nh; i++) {
        printf("H%d\t%d\t\t%d\n",
            holes[i].id,
            holes[i].original_size,
            holes[i].remaining_size);
    }

    return 0;
}
```

Output:

```
Enter number of holes: 5
Enter size of Hole 1: 100
Enter size of Hole 2: 500
Enter size of Hole 3: 200
Enter size of Hole 4: 300
Enter size of Hole 5: 600
```

```
Enter number of processes: 4
Enter memory requirement of Process 1: 212
Enter memory requirement of Process 2: 417
Enter memory requirement of Process 3: 112
Enter memory requirement of Process 4: 426
```

```
----- BEST FIT ALLOCATION -----

Process Size    Hole    Status
P1  212 H4  Allocated
P2  417 H2  Allocated
P3  112 H3  Allocated
P4  426 H5  Allocated
```

```
----- HOLE INFORMATION -----
Hole    Original    Remaining
H1  100     100
H3  200     88
H4  300     88
H2  500     83
H5  600     174
```

Table:

First Fit	Best Fit
Selects the first suitable hole	Selects the smallest suitable hole
Searches from beginning	Holes are sorted/considered by size
Faster/simple implementation	May require sorting/searching
Does not necessarily minimize leftover hole for each allocation	Produces smaller leftover holes

Conclusion:

Thus, the First Fit and Best Fit memory allocation strategies were implemented successfully. First Fit allocates a process to the first available hole that is large enough, while Best Fit allocates it to the smallest suitable hole. The experiment helped in understanding how different memory management strategies allocate memory to processes and how the remaining memory in each hole changes after allocation.

Signature of faculty in-charge